Contents

1 Connect4 notes **2**

1.1 Representing the game board 2

2 Function specifications **2**

2.1 Testing 2

2.2 `check_move(board, turn, col, pop)` 3

2.3 `apply_move(board, turn, col, pop)` 3

2.4 `check_victory(board, who_played)` 4

2.5 `computer_move(board, turn, level)` 4

2.6 `display_board(board)` 4

2.7 `menu()` 5

1 Connect4 notes

1.1 Representing the game board

The game board is represented as a list of $\text{rows} \times 7$ integers, consisting of only the values 0, 1, and 2.

- Zeros (`0`) in the list represent a blank tile, i.e. an unplayed tile.
- Ones (`1`) in the list represent a tile for player 1.
- Twos (`2`) in the list represent a tile for player 2.

The position of each integer in the list represents a tile in a fixed location in the board. In a board of size n, the first 7 entries represent the *bottom-most* row, the next 7 entries represent the second row, and so on, until the $\frac{n}{7}$-th set of 7 entries, representing the $\frac{n}{7}$-th row.

For example, in a board with 6 rows there are 42 integers, the first 7 represent the **bottom-most** row, the next set of 7 represent the row above that, and so on. The board `[1, 2, 0, 1, 2, 1, 1, 2, 2, 0, 2, 1, 0, 0]` represents the board state below, with player 1's tiles (arbitrarily) represented with an O, and player 2's tiles (arbitrarily) represented with an X.

X	X		X	O		
O	X		O	X	O	O

2 Function specifications

The functions must be implemented **exactly as per the specifications below**. If they do not meet the specifications, you will not have a good time. **Your functions do not meet the specifications** if:

- you modify the function inputs in any way such that the inputs and outputs are not exactly as described; or
- your function relies on other variables that are not in its input parameters; or
- your function relies on additional input from the user apart from its input parameters (i.e. any `input()` calls).

You may assume that the input is exactly as described, and that only sensible inputs are given. Input validation within each function is not necessary, but is easy enough to implement.

The gameplay rules are not described here.

2.1 Testing

To see if your functions are implemented correctly, you run the `test.py` file as an indicator. Before running it; ensure:

1. Any `input()` calls are within the `menu()` function only. In general, all code should be within a function.
2. Your `test.py` and `connect4.py` to be tested are in the same folder.
3. The last two lines of `connect4.py` file are below, and this should be the **only code** which is not in a function.

```
if __name__ == "__main__":
    menu()
```

If any **errors** (e.g. `SyntaxError`) occur, there is an issue in your code. If the code prints that there is an error *in a test*, your function has failed to produce the expected output given the inputs. It acts as a good indicator of how well you will score for the project, but does not include many edge cases.

2.2 check_move(board, turn, col, pop)

Inputs:

- `board` - a list of size $7 \times r$, consisting of only the values 0, 1, and 2, representing the state of the board.
- `turn` - an integer, either 1 or 2, representing whose turn is being played.
- `col` - an integer between 0 and 6, inclusive of both values, representing the column at which the player's move was made.
- `pop` - a boolean, representing whether the move is popping a tile out or not.

Outputs:

A boolean, `True` or `False`, depending on whether the move (`col` and `pop`) played by player `turn` with a board state `board` was a legal move or not, according to the gameplay rules.

2.3 apply_move(board, turn, col, pop)

Inputs:

- `board` - a list of size $7 \times r$, consisting of only the values 0, 1, and 2, representing the state of the board.
- `turn` - an integer, either 1 or 2, representing whose turn is being played.
- `col` - an integer between 0 and 6, inclusive of both values, representing the column at which the player's move was made.
- `pop` - a boolean, representing whether the move is popping a tile out or not.

The move being played, `col` and `pop`, are always legal moves according to the gameplay rules.

Outputs:

A list of size $7 \times r$, consisting of only the values 0, 1, and 2, representing the state of the board, after the move (`col` and `pop`) are played by player `turn` with board state `board`.

2.4 check_victory(board, who_played)

Inputs:

- `board` - a list of size $7 \times r$, consisting of only the values 0, 1, and 2, representing the state of the board.
- `who_played` - an integer, either 1 or 2, representing whose turn was played leading to the current state of the board.

Outputs:

An integer, `0` if there is no winner, 1 if player 1 wins, 2 if player 2 wins, according to the gameplay rules. If both players have a winning condition, then the player that allows their opponent to have a winning condition loses. e.g. player 1 makes a move that leads to both players winning - then player 1 loses (and player 2 wins).

2.5 computer_move(board, turn, level)

Inputs:

- `board` - a list of size $7 \times r$, consisting of only the values 0, 1, and 2, representing the state of the board.
- `turn` - an integer, either 1 or 2, representing the player number the computer is to make a move for.

Outputs:

A tuple of two items representing a valid move, ordered as:

1. An integer between 0 and 6, inclusive of both ends, representing the position at which the computer's move is to be played.
2. A boolean, representing whether the move is popping a tile out or not.

For `level = 1`, the returned move should be a valid move. You should consider using the `check_move()` you wrote.

For `level = 2`, the returned move has the following priorities:

1. If a winning move is available, take it.
2. Do not make a move that would either cause a win for the opponent, or cause a state of the board that allows the opponent to make a winning move.
3. Make a random move.

You should consider using the `check_move()`, `apply_move()`, and `check_victory()` functions you wrote.

2.6 display_board(board)

Inputs:

- `board` - a list of size $7 \times r$, consisting of only the values 0, 1, and 2, representing the state of the board.

Outputs:

The function **does not return anything**. It displays the given board to the user.

2.7 `menu()`

Inputs: none.

Outputs: none.

Its purpose is meant to act as the main gameplay loop, taking user input and directing how the game flows. User interaction should not be outside this function.